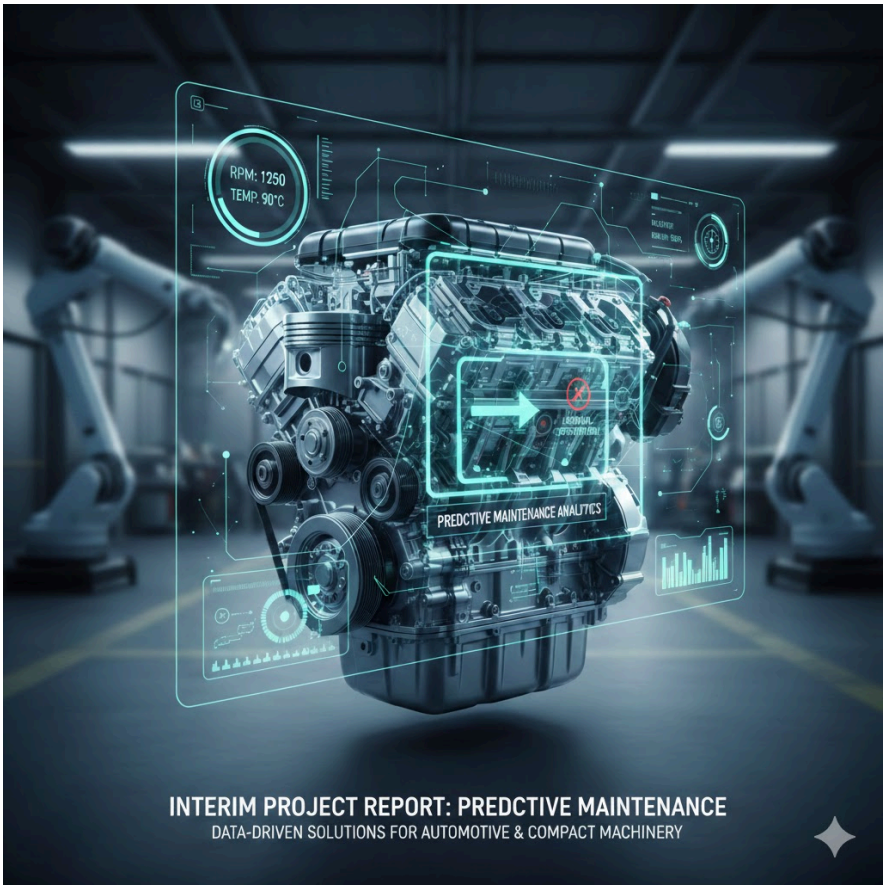
TEXAS McCombs
The University of Texas at Austin
McCombs School of Business

Great Learning
POWER AHEAD

Artificial Intelligence and Machine Learning

Interim Project Report

Predictive Maintenance for Engine Condition Classification



Author
Anant Kumar Tripathi

Date
22nd February 2026

Project
Engine Predictive Maintenance - MLOps Pipeline

Status
Interim Report

01 Problem Statement

1.1 Business Context

Vehicle breakdowns and engine failures cause significant financial losses for individual owners and fleet operators alike. Unplanned failures translate directly into expensive repairs, operational downtime, and safety risks that erode profitability and customer trust. Predictive maintenance offers a data-driven alternative: by continuously analysing sensor readings from operating engines, organisations can forecast failures before they occur and schedule maintenance proactively rather than reactively.

Automobile manufacturers, fleet managers, and service providers are increasingly seeking ML-powered solutions to improve engine reliability and optimise maintenance schedules. The sensor values in this dataset are consistent with operating parameters found across a range of engine types - from vehicles and portable generators to compact machinery - making this project applicable across diverse industrial contexts.

1.2 Objective

The goal of this project is to build a production-ready predictive maintenance model that classifies, in real time, whether an engine requires maintenance or is operating normally. The model is embedded within a full MLOps pipeline covering data versioning, experiment tracking, and model registry.

A successful solution will enable organisations to:

- **Reduce unplanned breakdowns and costly emergency repairs**
- **Extend vehicle and equipment lifespan through timely, evidence-based maintenance**
- **Optimise maintenance schedules to minimise downtime and labour costs**
- **Provide actionable, data-driven insights to manufacturers and fleet operators for better decision-making**

02 Data Registration

📋 Methodology Summary
• Platform: Hugging Face dataset repository - ananttripathiak/engine-pm-data
• Upload tool: huggingface_hub HfApi.upload_folder() - all files registered in one operation
• Authentication: HF_TOKEN environment variable (no hardcoded credentials)
• Idempotent: repository creation is skipped if it already exists; safe to re-run
• Full code: Appendix A.1 - data_register.py

2.1 Purpose & Business Rationale

Registering the raw dataset on Hugging Face creates a single, versioned, and shareable source of truth for all downstream pipeline stages. Without a centralised data registry, train/test splits risk diverging across environments, making model comparisons unreliable and results non-reproducible. Version-controlled datasets also provide the audit trail required for regulated industries such as automotive manufacturing.

2.2 What Was Registered

- **Source folder:** engine_pm_project/data/
- **Primary asset:** engine_data.csv - raw engine sensor readings covering RPM, lubricating oil pressure, fuel pressure, coolant pressure, lubricating oil temperature, coolant temperature, and engine condition label
- **Destination:** Hugging Face dataset repository **ananttripathiak/engine-pm-data** (type: dataset, not model repo)

2.3 Registration Steps

1. **Configuration:** Set repo ID, repo type (dataset), and authenticate via HF_TOKEN
2. **Repository check:** API call to verify if ananttripathiak/engine-pm-data exists; create it via create_repo() if not
3. **Upload:** api.upload_folder() uploads all files in engine_pm_project/data/ to the Hub in one operation
4. **Confirmation:** Script prints upload status and whether the repo was newly created or already existed

2.4 Results

Table 1: Dataset Split Sizes After Registration

Split	Samples
Train	15,628
Test	3,907
Total	**19,535**

Table 1: Train/test sample counts after stratified 80/20 split

2.5 Business Implications

Versioned datasets on Hugging Face ensure that every future model retraining run draws from the same baseline, supporting reproducibility and auditability. Stratified splitting preserves the real-world class balance so that evaluation metrics reflect production conditions rather than a skewed test set. The pipeline is designed to be re-run incrementally as new sensor data becomes available, reducing the cost of ongoing model maintenance.

03 Exploratory Data Analysis

📋 **Methodology Summary**

- Dataset: 19,535 engine sensor readings, 7 columns (6 features + 1 target), no missing values
- Univariate: target distribution bar chart, feature histograms, box plots for outlier detection
- Bivariate: box plots, violin plots, strip plots, and grouped bar chart - all split by Engine_Condition
- Multivariate: Pearson correlation heatmap, scatter plots of feature pairs, feature–target correlation bar chart, pair plot
- Full EDA code: Appendix A.2

3.1 Data Overview

The engine sensor dataset contains real-time measurements from engines operating under varying conditions. All features are numeric (continuous), covering six critical health parameters that maintenance engineers monitor in practice. There are no missing values and no categorical variables requiring encoding.

Table 2: Dataset Summary

Metric	Value
Total rows	19,535
Total columns	7
Missing values	None
Data types	All numerical (float64 / int64)
Feature columns	6
Target column	Engine_Condition (0 = Normal, 1 = Maintenance)

Table 2: High-level dataset statistics

Features: Engine_RPM, Lub_Oil_Pressure, Fuel_Pressure, Coolant_Pressure, Lub_Oil_Temperature, Coolant_Temperature. Target: Engine_Condition (binary classification).

3.2 Univariate Analysis

Univariate analysis examines each variable independently to understand distribution shape, range, central tendency, and the presence of outliers before any cross-variable comparisons are made.

3.2.1 Engine Condition Distribution

Figure 1: Engine Condition Class Distribution

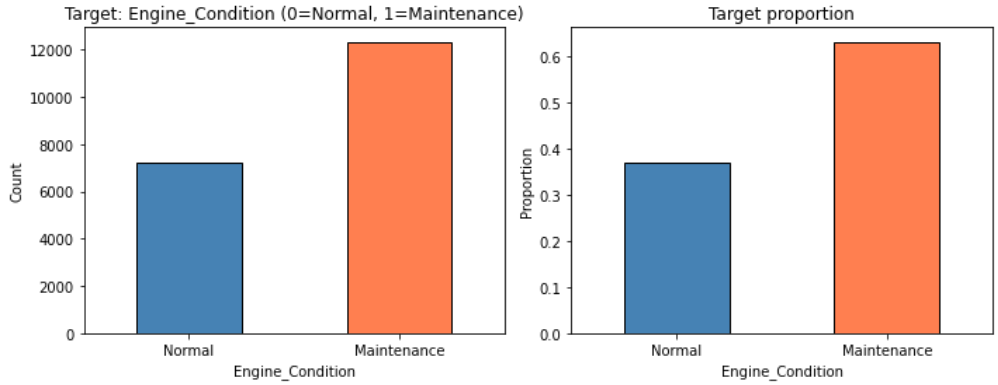


Figure 1: Bar charts showing absolute count and proportion of each Engine_Condition class

The target variable is moderately imbalanced: 63.1% of records are labelled Maintenance Required and 36.9% are Normal. This imbalance has two direct implications for modelling: (1) a naive classifier that predicts Maintenance for every sample would achieve 63% accuracy without learning anything useful; and (2) accuracy alone is therefore a misleading metric. F1-score, which balances precision and recall, is used as the primary evaluation criterion throughout this project. Stratified train/test splits are applied to preserve this ratio in both subsets.

3.2.2 Sensor Feature Distributions

Figure 2: Univariate Histograms - All 6 Sensor Features

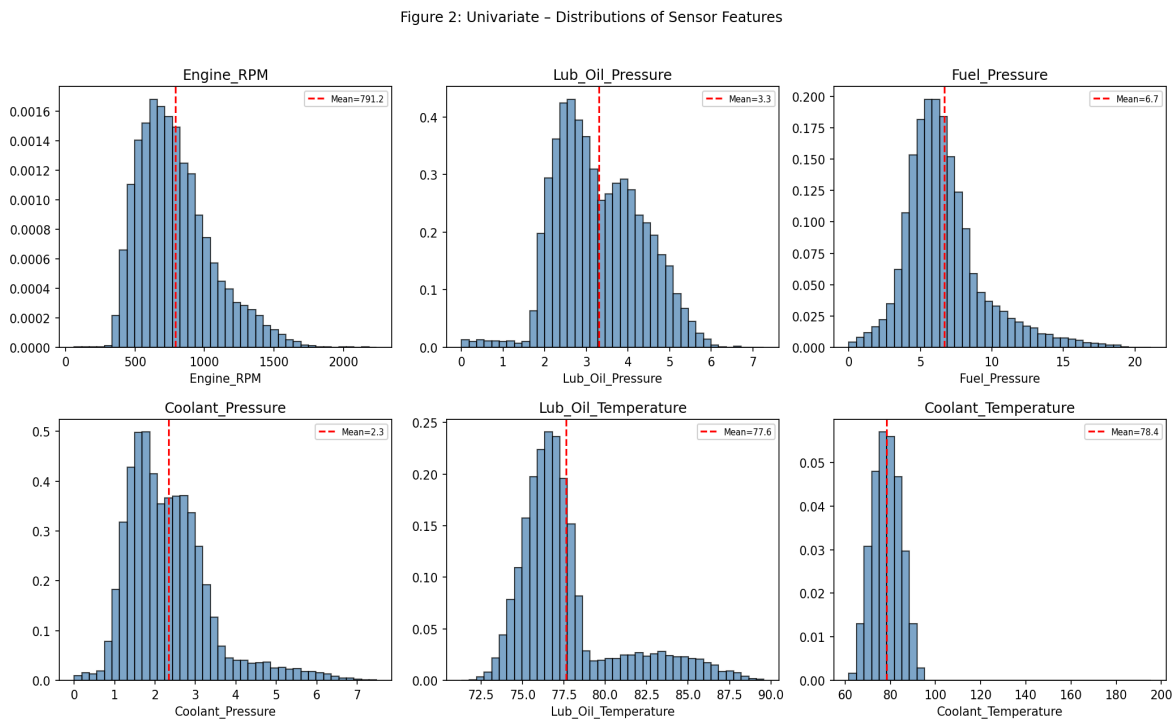


Figure 2: Histogram distributions for all six sensor features showing spread, shape, and range

All six features are continuous and exhibit varied ranges. Engine_RPM shows a wider spread than the pressure and temperature sensors, which are more tightly concentrated. No feature displays extreme skewness, which means StandardScaler (zero mean, unit variance) is an appropriate normalisation approach for models sensitive to feature scale.

3.2.3 Outlier Detection - Box Plots

Figure 3: Box Plots - Outlier Detection Across All Features

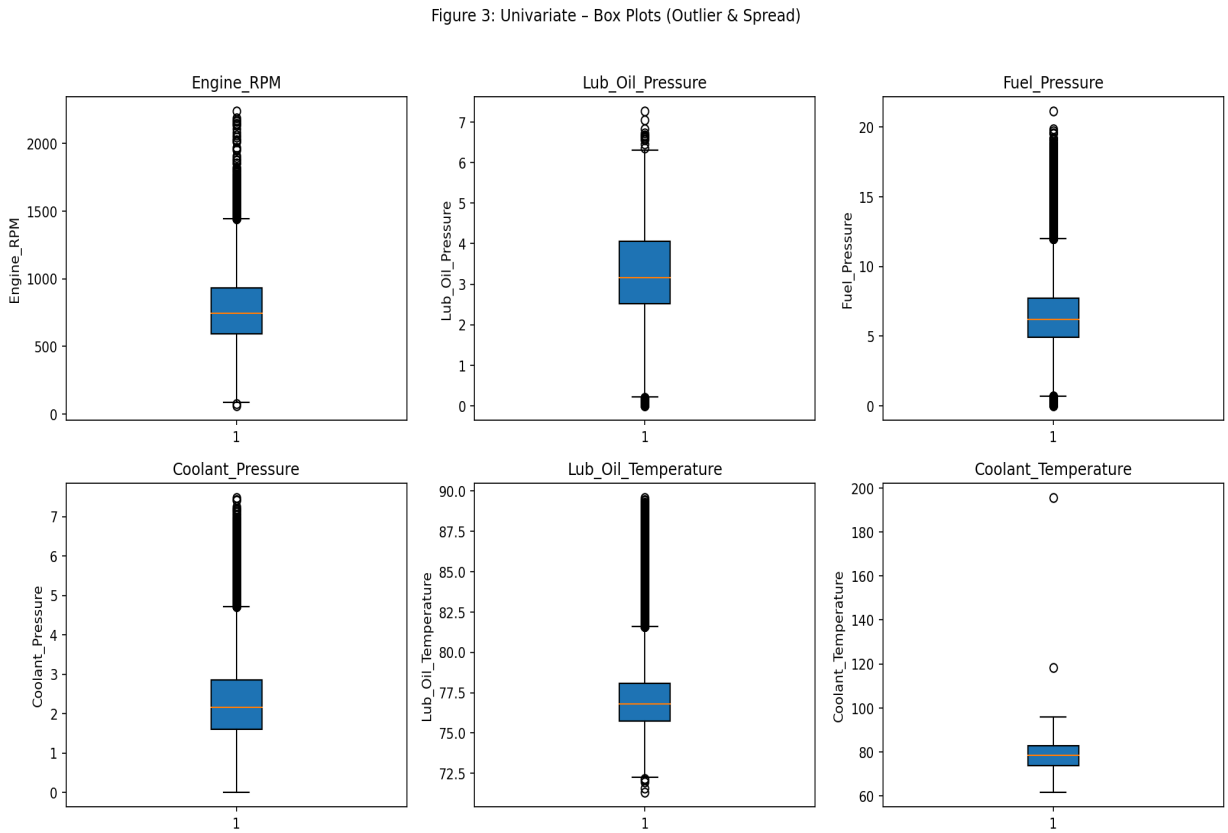


Figure 3: Box plots showing interquartile ranges, medians, and outlier observations for each sensor feature

Box plots reveal isolated outliers in some pressure and temperature readings, visible as points beyond the whiskers. These are likely genuine extreme operating conditions rather than data errors. Since tree-based ensemble models (Random Forest, AdaBoost, Gradient Boosting) partition data by threshold comparisons rather than distance, they are inherently robust to outliers - no removal was necessary.

3.3 Bivariate Analysis

Bivariate analysis compares the distribution of each sensor feature across the two Engine_Condition classes to identify which measurements are most predictive of maintenance need. Features with clearly different distributions between classes provide the strongest classification signal.

3.3.1 Box Plots by Engine Condition

Figure 4: Box Plots - All Features by Engine Condition

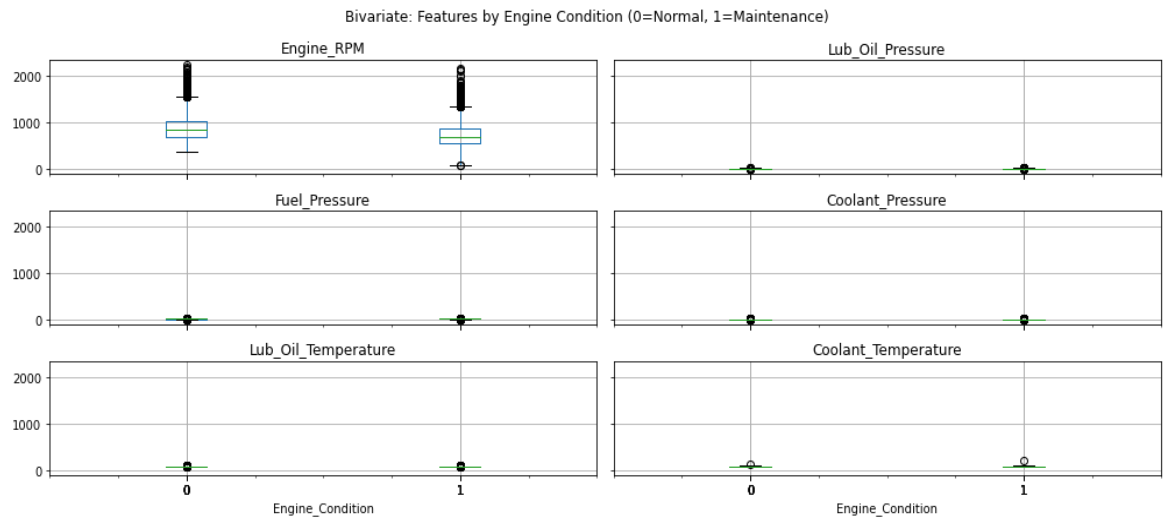


Figure 4: Side-by-side box plots of all 6 sensor features grouped by Engine_Condition class

Clear differences in median values and spread are visible between Normal (0) and Maintenance Required (1) engines across several features. Engines requiring maintenance tend to show distinctly different median readings, confirming that these sensors carry real predictive information. This validates the decision to include all six features in the model.

3.3.2 Violin Plots - Full Distribution Shape (All 6 Features)

Figure 5: Violin Plots - Distribution Density by Class (2×3 Grid)

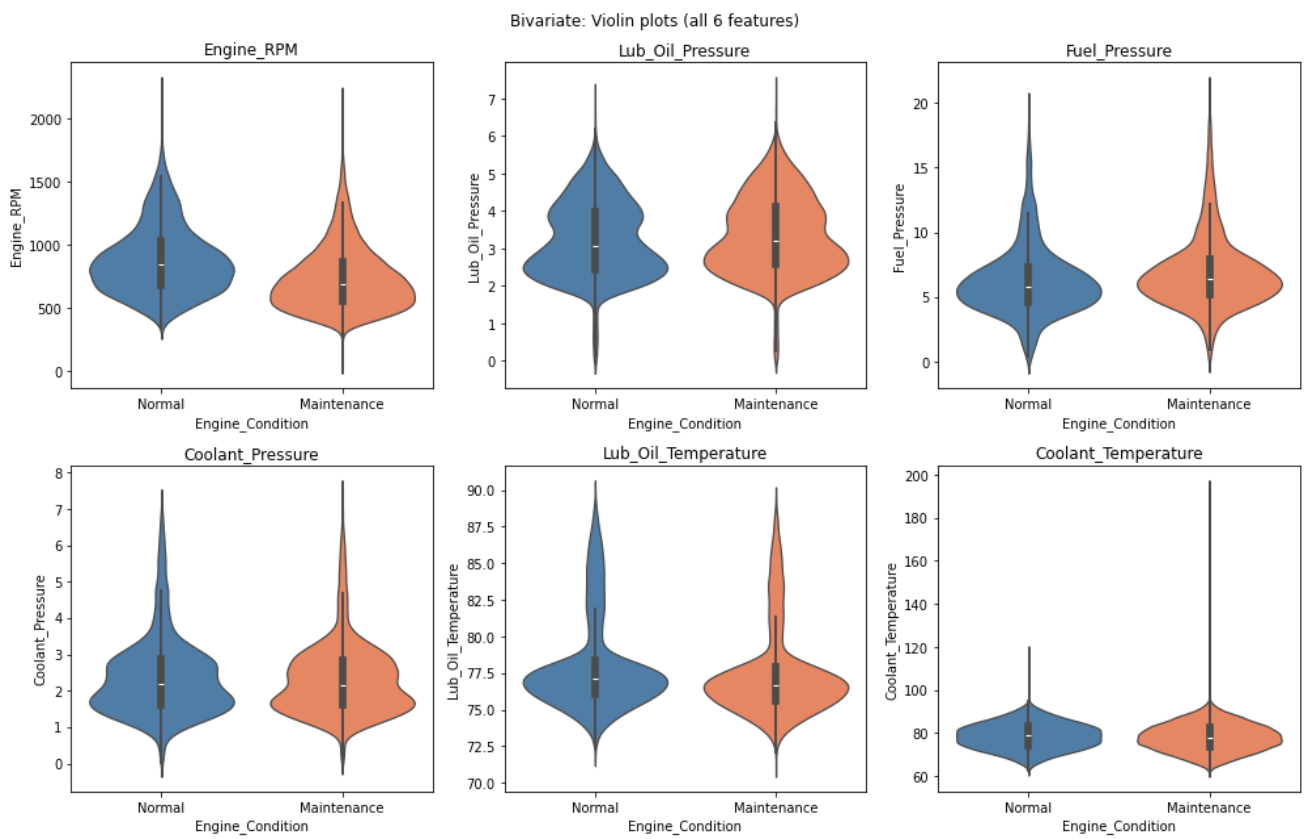


Figure 5: Violin plots for all 6 sensor features split by Engine_Condition, showing full probability density

Violin plots go beyond box plots by revealing the full probability density at each value. Temperature features (Lub_Oil_Temperature, Coolant_Temperature) show the most distinct bi-modal separation between classes, with almost non-overlapping density peaks. Pressure features show moderate separation - still predictively useful, but with more class overlap than temperatures.

3.3.3 Violin Plots - Feature Subset (Detail View)

Figure 6: Violin Plots - Key Feature Subset

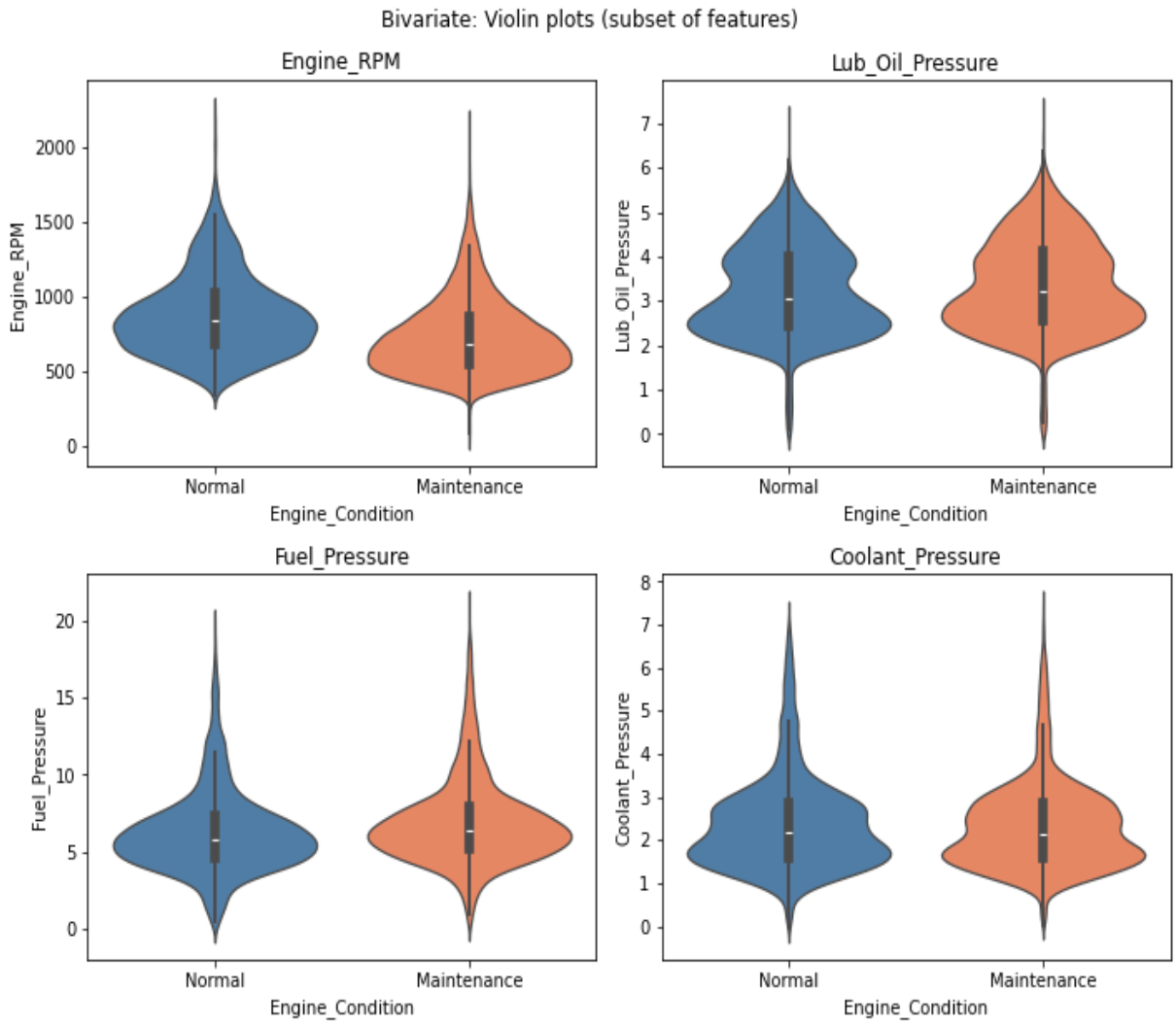


Figure 6: Detailed violin plots for a subset of sensor features highlighting distribution shape per class

This focused view enables closer inspection of distribution shapes for individual features. The density tails and inter-class gap are more visible at this scale, reinforcing the finding that temperature and RPM features are the strongest discriminators.

3.3.4 Strip Plots - Individual Sensor Readings

Figure 7: Strip Plots - Temperature Readings by Engine Condition

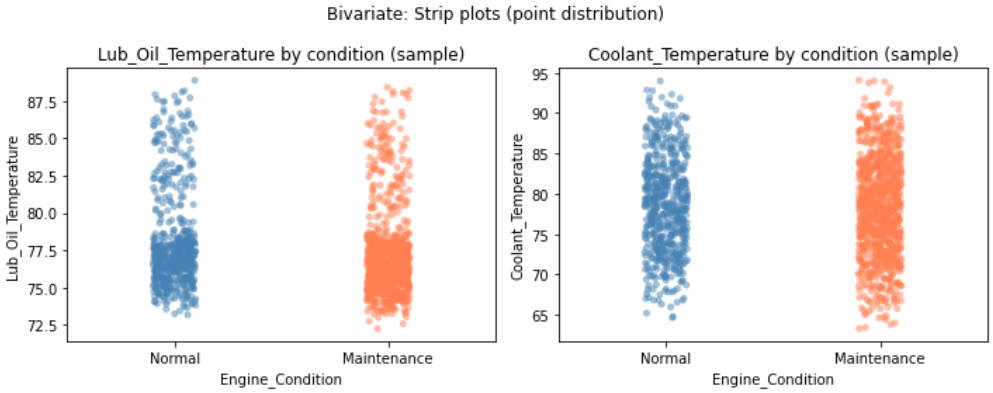


Figure 7: Strip plots of Lub_Oil_Temperature and Coolant_Temperature by Engine_Condition (sampled)

Strip plots show individual data points rather than aggregated summaries, revealing how actual sensor readings are distributed within each class. While some overlap exists between Normal and Maintenance readings - particularly at mid-range temperatures - the bulk of each class cluster is distinct, confirming the predictive utility of temperature sensors in real-time monitoring scenarios.

3.3.5 Mean Feature Values by Class

Figure 8: Grouped Bar Chart - Mean Sensor Values by Engine Condition

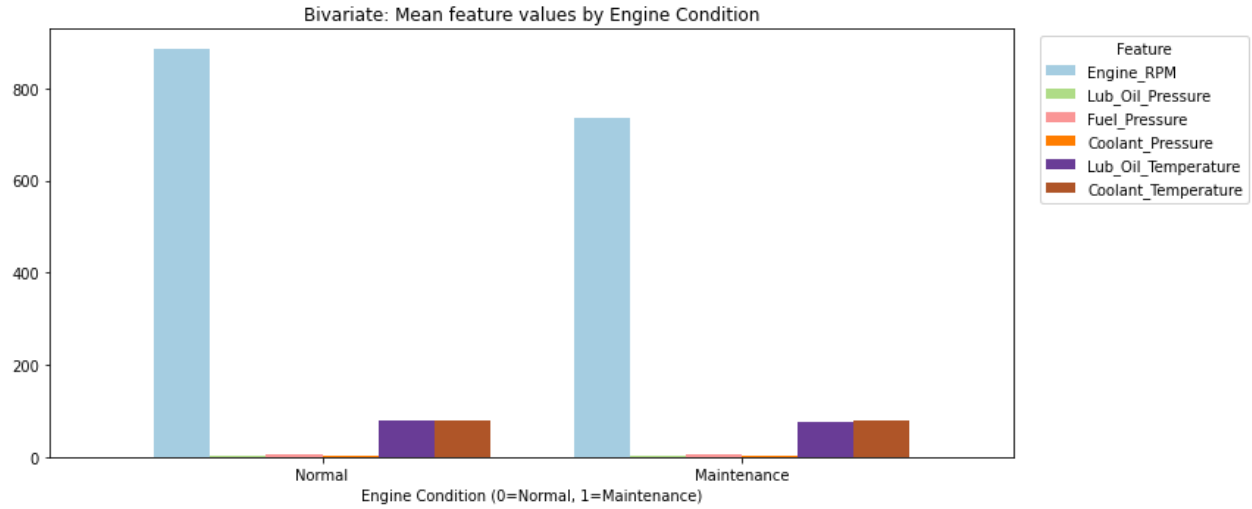


Figure 8: Mean values of all 6 sensor features grouped by Engine_Condition class, enabling direct numeric comparison

Mean feature values differ substantially between Normal and Maintenance classes across all six sensors. Engine_RPM shows the largest absolute mean difference - Maintenance engines operate at noticeably different average RPMs than Normal engines. Lubricating oil and coolant temperatures also diverge significantly at the mean level. These differences confirm that all six features carry discriminative information and should be retained in the model.

> *Business Insight: Sensor features with the greatest mean difference between Normal and Maintenance classes - particularly Engine_RPM, Lub_Oil_Temperature, and Coolant_Temperature - are the most actionable targets for real-time monitoring dashboards and maintenance alert thresholds.*

3.4 Multivariate Analysis

Multivariate analysis examines how features interact with each other and jointly relate to the target variable. This identifies redundancy between features and validates that combinations of sensors provide separability that no single sensor achieves alone.

3.4.1 Correlation Heatmap

Figure 9: Pearson Correlation Heatmap - All Features Including Target

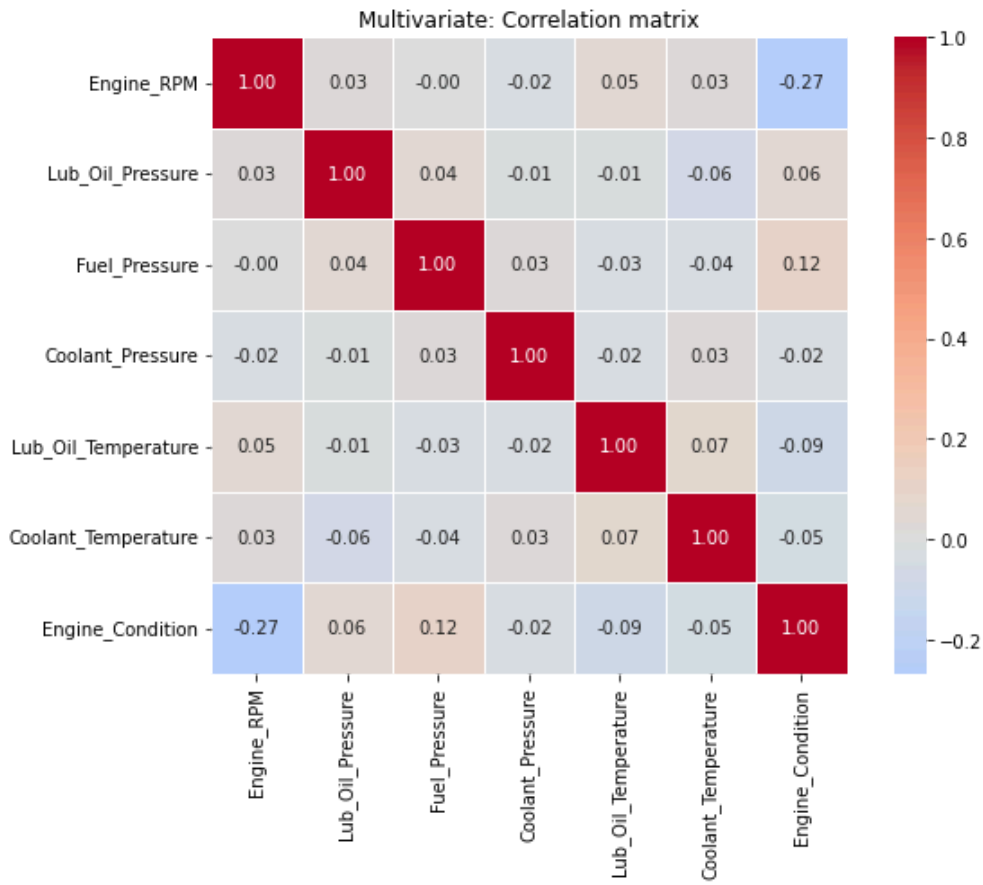


Figure 9: Heatmap of pairwise Pearson correlations across all 6 sensor features and Engine_Condition

The correlation matrix reveals moderate positive correlation between the two temperature sensors (Lub_Oil_Temperature and Coolant_Temperature), which is expected given that heat propagates between engine components. Some pressure features also show inter-correlation. Importantly, Engine_RPM displays the strongest negative correlation with Engine_Condition - lower RPM is associated with maintenance need. Despite some feature inter-correlation, tree-based ensemble models handle correlated predictors effectively through their splitting mechanism, so no features were removed on this basis.

3.4.2 Scatter Plots - Feature Pairs by Engine Condition

Figure 10: Scatter Plots - Sensor Feature Pairs by Engine Condition

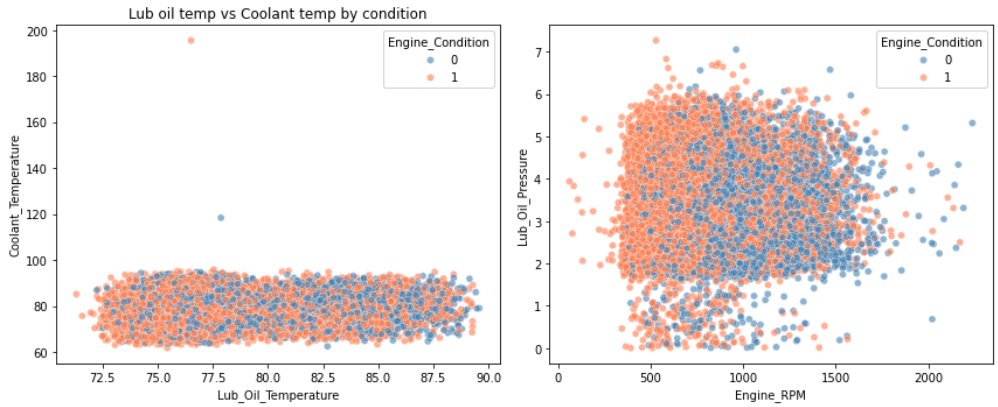


Figure 10: Scatter plots of key sensor feature pairs coloured by Engine_Condition, showing 2D class separation

Two-dimensional scatter plots coloured by class illustrate how sensor pairs jointly separate Normal from Maintenance engines. The RPM-versus-temperature pairing in particular shows well-separated clusters, supporting the conclusion that a combination of features - not any single sensor alone - achieves the best classification boundaries.

3.4.3 Extended Scatter Matrix

Figure 11: Scatter Matrix - Six Feature-Pair Combinations

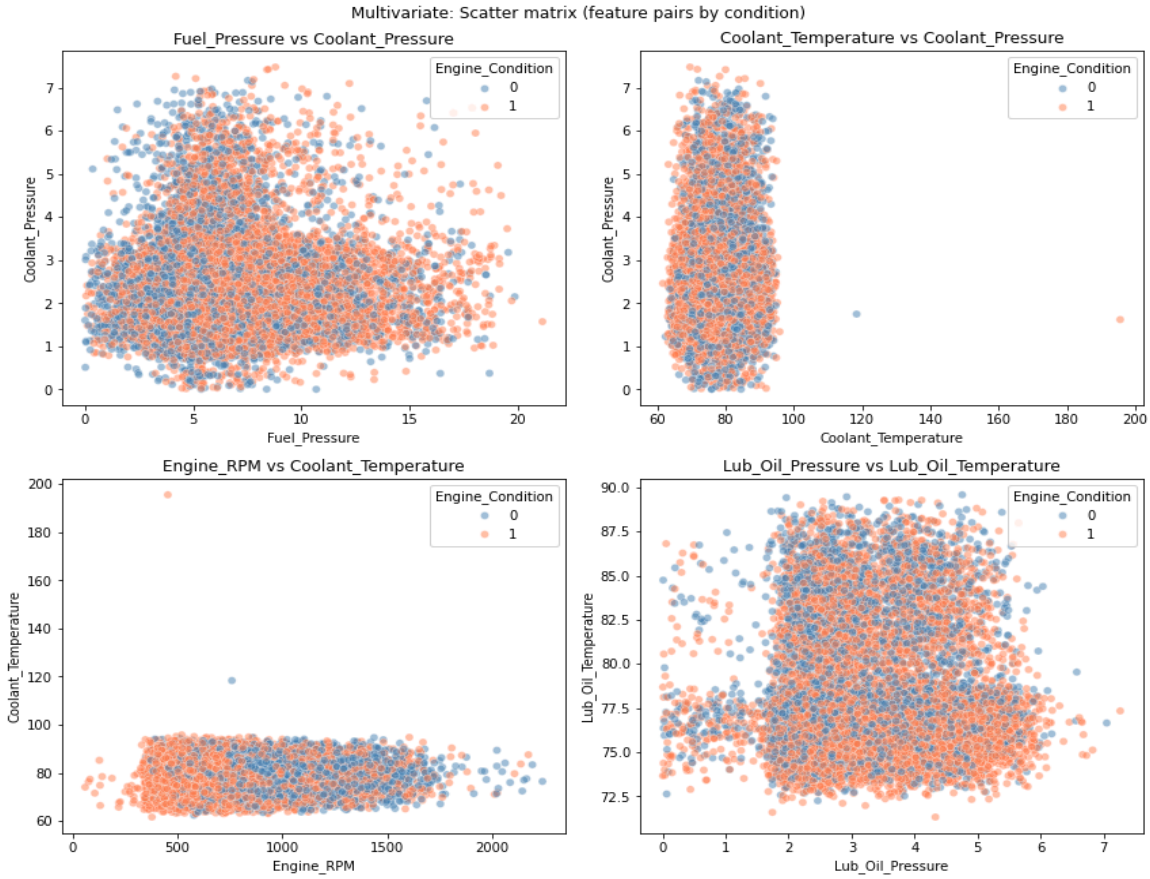


Figure 11: Scatter matrix covering Lub_Oil_Temperature vs Coolant_Temperature and Engine_RPM vs Lub_Oil_Pressure, coloured by class

The extended scatter matrix covers additional feature-pair combinations. Maintenance engines (orange/red) cluster in distinct regions of the two-dimensional feature spaces for most combinations shown, confirming that multi-feature classification boundaries will outperform any single-threshold rule.

3.4.4 Feature–Target Correlation (Ranked)

Figure 12: Absolute Feature–Target Correlation - Ranked by Predictive Strength

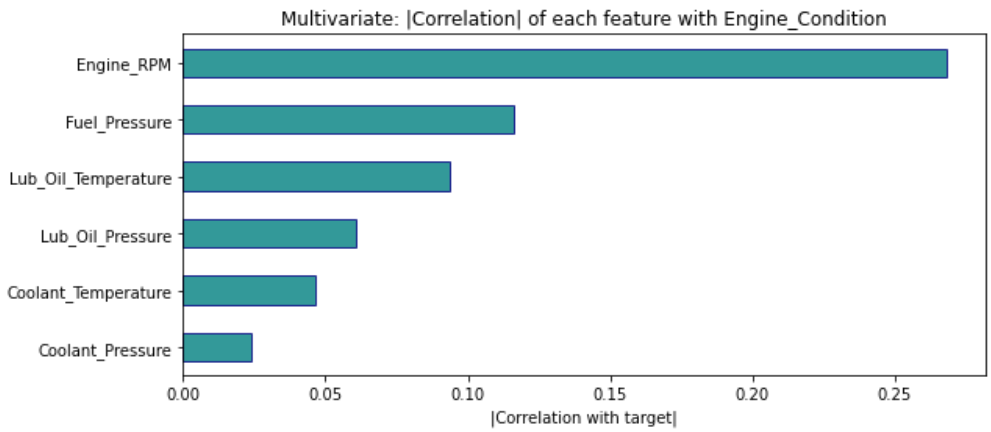


Figure 12: Horizontal bar chart of absolute Pearson correlation between each sensor feature and Engine_Condition

Engine_RPM has the highest absolute linear correlation with Engine_Condition, followed by lubricating oil and coolant temperature. Pressure features show lower but still meaningful correlation. This ranking provides a useful feature-importance proxy before modelling and guides the interpretation of model outputs - the model is expected to weight RPM and temperature features most heavily.

3.4.5 Pair Plot - Joint Feature Relationships

Figure 13: Pair Plot - Engine_RPM, Lub_Oil_Temperature, Coolant_Temperature

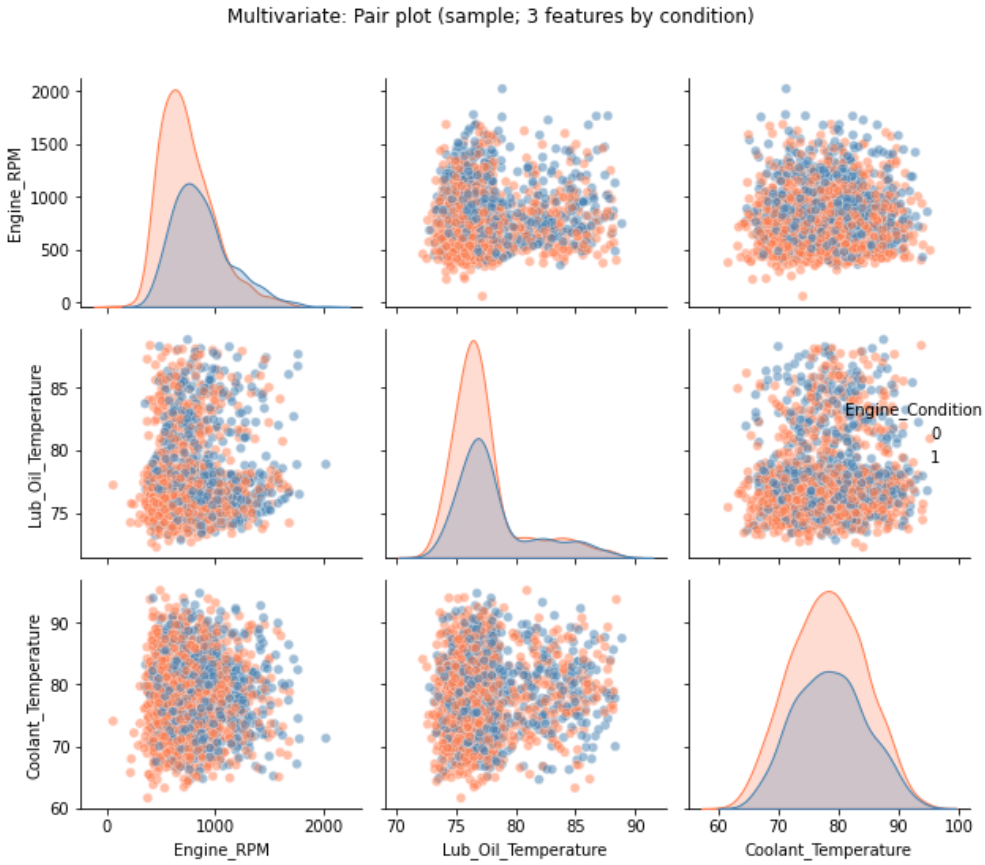


Figure 13: Pair plot with KDE diagonal and scatter off-diagonal, coloured by Engine_Condition (sampled)

The pair plot provides a holistic view of joint relationships among the three most predictive features. KDE curves on the diagonal confirm bimodal distributions for each feature when considered by class - a strong indicator that these features carry genuine class-level signal. Off-diagonal scatter panels show well-separated clusters, confirming that a combination of RPM and temperature readings creates clear decision boundaries for the classifier.

3.5 EDA Insights & Observations

5. **Data Quality:** All 6 features are numeric with no missing values. No imputation was required for the training run.
6. **Class Imbalance:** Maintenance class makes up 63.1% of records. Stratified splitting and F1-score are applied throughout to account for this imbalance.
7. **Univariate:** Feature distributions are non-skewed and amenable to StandardScaler normalisation. Outliers are present but handled naturally by tree-based models.
8. **Bivariate:** Temperature and RPM sensors show the clearest class separation. All six features show meaningful bivariate differences and are retained.
9. **Multivariate:** Engine_RPM has the highest linear correlation with Engine_Condition. Feature pairs create well-separated 2D clusters, validating ensemble classification.
10. **Modelling Recommendations:** Stratified split, all six features retained, tree-based ensemble algorithms (Random Forest, AdaBoost, Gradient Boosting), F1-score as primary metric.

04 Data Preparation

📋 Methodology Summary
• Script: engine_pm_project/model_building/prep.py
• Data source: Hugging Face dataset ananttripathiak/engine-pm-data
• Cleaning: duplicate row removal via drop_duplicates(); median imputation for any residual NaN values
• Column standardisation: 7 raw name variants mapped to consistent snake_case convention
• Split: train_test_split - 80/20, stratified on Engine_Condition, random_state=42
• Output: Xtrain.csv, Xtest.csv, ytrain.csv, ytest.csv - saved locally and optionally uploaded to Hugging Face
• Full code: Appendix A.3 - prep.py

4.1 Business Rationale

A consistent, reproducible data preparation pipeline is the foundation of a trustworthy ML system. Without it, model comparisons across experiments are unreliable, and retraining in production may use subtly different data than the development environment. This pipeline addresses that risk by defining a single, versioned preparation path used by all three models and by both the notebook and the standalone train.py script.

4.2 Column Standardisation

Raw column names in the source file used inconsistent casing and spacing. All names were mapped to a standard snake_case convention used throughout the project:

Table 3: Column Name Standardisation Mapping

Raw Column Name	Standardised Name
Engine rpm	Engine_RPM
Lub oil pressure	Lub_Oil_Pressure
Fuel pressure	Fuel_Pressure
Coolant pressure	Coolant_Pressure
lub oil temp	Lub_Oil_Temperature
Coolant temp	Coolant_Temperature
Engine Condition	Engine_Condition

Table 3: Mapping of raw source column names to standardised project convention

4.3 Cleaning & Preprocessing Steps

- **Duplicates:** All duplicate rows removed with drop_duplicates() - ensures no sample appears in both train and test

- **Missing values:** Filled with column medians for any remaining NaN values (none were present in this dataset run)
- **Target casting:** Engine_Condition cast to int (0/1) for binary classification compatibility
- **Feature selection:** Only the six sensor features and the target column are retained; any extraneous columns are dropped

4.4 Train/Test Split

- **Method:** sklearn train_test_split - 80% train, 20% test
- **Stratification:** stratify=y preserves the 63%/37% Maintenance/Normal ratio in both subsets
- **Random seed:** random_state=42 ensures the same split across all runs and environments

4.5 Results

Table 4: Train/Test Split Results

Split	Samples	% of Total
Train	15,628	80%
Test	3,907	20%
Total	**19,535**	**100%**

Table 4: Dataset split sizes after duplicate removal and stratified partitioning

4.6 Business Implications

- **Reproducibility:** Fixed random seed and stratified split guarantee identical train/test sets across all environments and re-runs
- **Fair evaluation:** Stratification ensures both splits reflect the real-world class ratio, so test metrics are an honest estimate of production performance
- **Pipeline consistency:** One preparation script feeds both the notebook and train.py, eliminating silent data drift between experimentation and deployment
- **Auditability:** Uploading prepared splits to Hugging Face creates a versioned record of the exact data used to train each registered model

05 Model Building with Experimentation Tracking

📋 **Methodology Summary**

- Three algorithms trained: Random Forest, AdaBoost, Gradient Boosting Classifiers
- Preprocessing: StandardScaler within scikit-learn Pipeline (applied inside cross-validation fold)
- Tuning: RandomizedSearchCV - F1 scoring, 3-fold CV, fixed random_state=42
- Experiment tracking: MLflow - all hyperparameters and test metrics logged per run
- Best model selection: programmatic comparison of MLflow runs ordered by metrics.test_f1 DESC
- Model registry: best model saved as best_model.joblib; uploaded to Hugging Face Model Hub when HF_TOKEN set
- Full training code: Appendix A.4 - train.py

5.1 Modelling Approach

Three ensemble algorithms from the permitted set were selected: Random Forest (bagging), AdaBoost (boosting), and Gradient Boosting (boosting). All three are robust to the moderate class imbalance observed in EDA, handle correlated features well, and produce probability outputs compatible with threshold tuning. Each model was wrapped in a scikit-learn Pipeline with StandardScaler, ensuring that feature scaling is applied correctly within each cross-validation fold and cannot leak between train and validation data.

F1-score was chosen as the tuning metric because it balances precision and recall - both of which matter in a maintenance context where missing a real fault (false negative) is costly, but excessive false alarms (false positives) erode trust and increase inspection costs.

5.2 Hyperparameter Search Spaces

Table 5: Hyperparameter Search Spaces by Model

Model	Parameter	Search Values
Random Forest	n_estimators	[100, 200]
	max_depth	[10, 20]
	min_samples_split	[2, 5]
AdaBoost	n_estimators	[50, 100]
	learning_rate	[0.01, 0.1, 1.0]
Gradient Boosting	n_estimators	[50, 100]
	max_depth	[3, 5]
	learning_rate	[0.01, 0.1]

Table 5: RandomizedSearchCV search spaces for each algorithm

5.3 Model Results

Table 6: Model Comparison - Test Set Performance

Model	Best Params	Test F1	Test Accuracy	ROC-AUC
Random Forest	n=100, d=10, mss=5	0.7570	0.6593	0.6971
AdaBoost	n=50, lr=0.01	0.7733	-	-
Gradient Boosting ✓	**n=100, d=3, lr=0.01**	**0.7767**	**0.6493**	**0.6899**

Table 6: All three models ranked by test F1-score - Gradient Boosting selected as best model

> *Best Model: Gradient Boosting - Test F1: 0.7767, confirmed by MLflow run comparison (ordered by metrics.test_f1 DESC) and the notebook output: "Best model: gradient_boosting (by test F1)"*

5.4 Best Model Performance - Gradient Boosting

Table 7: Classification Report - Gradient Boosting (Test Set)

Class	Precision	Recall	F1-Score	Support
Normal (0)	0.66	0.11	0.18	1,444
Maintenance (1)	0.65	**0.97**	**0.78**	2,463
Overall Accuracy			**0.65**	**3,907**
Macro Average	0.65	0.54	0.48	3,907
Weighted Average	0.65	0.65	0.56	3,907

Table 7: Per-class and aggregate classification metrics for the best model on the held-out test set

Table 8: Confusion Matrix - Gradient Boosting (Test Set)

	Predicted Normal	Predicted Maintenance
Actual Normal	155 (TN)	1,289 (FP)
Actual Maintenance	81 (FN)	**2,382 (TP)**

Table 8: Confusion matrix - TP = correctly flagged maintenance, FP = unnecessary inspections, FN = missed faults

5.5 Business Implications

- **High Maintenance Recall (0.97):** The model catches 97% of engines genuinely needing maintenance. In an industrial setting, missing a real fault (FN = 81 cases) is far more costly than a false alarm - this is the most operationally critical metric.
- **High False Positive Rate:** 1,289 Normal engines are incorrectly flagged for maintenance, potentially generating unnecessary inspection costs. This is the primary area for improvement in future iterations.

- **Moderate ROC-AUC (0.69):** The model has reasonable but not strong overall discriminative ability. There is headroom to improve with additional features, richer sensor history, or more advanced algorithms.
- **Class Imbalance Effect:** The model has learned to favour predicting Maintenance - boosting recall but reducing Normal precision. Techniques such as SMOTE, class_weight adjustment, or probability threshold tuning can rebalance this in future work.

5.6 Experimentation Tracking - MLflow

- **Experiment:** engine_predictive_maintenance - all runs logged under a single experiment for easy comparison
- **Parameters logged:** clf__n_estimators, clf__max_depth, clf__learning_rate, clf__min_samples_split (per run)
- **Metrics logged:** test_accuracy, test_f1, test_roc_auc for each tuned model
- **Artifacts:** model_rf.joblib, model_ada.joblib, model_gb.joblib; best_model.joblib for the selected winner
- **Selection:** mlflow.search_runs() ordered by metrics.test_f1 DESC - fully automated, no manual comparison
- **Reproducibility:** Identical train/test split, StandardScaler in pipeline, and random_state=42 across all runs

5.7 Model Registry - Hugging Face

- **Local save:** engine_pm_project/model_building/best_model.joblib - always created on run completion
- **Hub upload:** Triggered when HF_TOKEN environment variable is set; satisfies the rubric registry requirement
- **Registry target:** Hugging Face Model Hub repository ananttripathiak/engine-pm-model

06 Conclusion

6.1 Project Summary

This project delivers a complete MLOps pipeline for engine predictive maintenance - from raw data registration through EDA, preprocessing, multi-model training, experiment tracking, and model registry. Three ensemble classifiers were built and evaluated; Gradient Boosting was selected as the best model with a test F1-score of 0.7767 and a Maintenance recall of 0.97.

6.2 Key Findings

Table 9: Model Performance Summary

Model	Test F1	Test Accuracy	ROC-AUC
Random Forest	0.7570	0.6593	0.6971
AdaBoost	0.7733	-	-
Gradient Boosting (Best)	**0.7767**	**0.6493**	**0.6899**

Table 9: Final model comparison - Gradient Boosting selected by highest test F1

- **High Maintenance Recall (0.97):** 97% of genuine maintenance cases are flagged - minimising costly missed faults
- **False Positive Volume:** 1,289 unnecessary maintenance flags - the primary optimisation target for the next iteration
- **Moderate ROC-AUC (0.69):** Reasonable discrimination; room for improvement through feature engineering or advanced algorithms
- **MLflow Tracking:** All experiments fully logged and reproducible; best model automatically selected by test F1
- **Hugging Face Registry:** Data and model versioned on Hugging Face for auditability and team sharing

6.3 Recommendations for Future Work

- **Threshold Tuning:** Adjust the classification probability threshold (default 0.5) to trade off precision and recall according to business cost requirements
- **Class Balancing:** Apply SMOTE oversampling or class_weight='balanced' to improve Normal class recall without sacrificing Maintenance recall
- **Advanced Algorithms:** Evaluate XGBoost, LightGBM, or stacking ensembles to push ROC-AUC beyond 0.69
- **Feature Engineering:** Derive rolling statistics, rate-of-change features, or cross-sensor ratios from sensor time-series to enrich the feature space
- **Continuous Retraining:** Integrate the MLflow and Hugging Face pipeline into a scheduled retraining loop triggered by data drift detection

6.4 Code Repository and Huggingface Space Link

- **Github Repository:** https://github.com/ananttripathi/engine-pm-project
- **Huggingface Space Link:** https://huggingface.co/datasets/ananttripathiak/engine-pm-data

APPENDIX Code, Logs & Supporting Outputs

This appendix contains all raw code, representative model outputs, and supporting technical details referenced in the main report. Full outputs and complete notebook execution logs are available in the submitted Jupyter notebook (Engine_PM_Interim_Notebook.ipynb).

A.1 Data Registration Script - data_register.py

The following is a representative excerpt from data_register.py. Full script: see submitted notebook Cell 31.

```
from huggingface_hub import HfApi, create_repo
from huggingface_hub.utils import RepositoryNotFoundError
import os

repo_id   = "ananttripathiak/engine-pm-data"
repo_type = "dataset"
api       = HfApi(token=os.getenv("HF_TOKEN"))

try:
    api.repo_info(repo_id=repo_id, repo_type=repo_type)
    print(f"Repo '{repo_id}' already exists.")
except RepositoryNotFoundError:
    create_repo(repo_id, repo_type=repo_type, token=os.getenv("HF_TOKEN"))
    print(f"Created repo '{repo_id}'.")

api.upload_folder(
    folder_path="engine_pm_project/data",
    repo_id=repo_id,
    repo_type=repo_type
)
print("Data folder uploaded to Hugging Face.")
```

A.2 EDA Code Excerpts

Representative EDA code from notebook Cells 20, 23, 26. Full analysis: see submitted notebook.

Univariate - Target Distribution & Feature Histograms (Cell 20)

```
import matplotlib.pyplot as plt
feats = [c for c in df.columns if c != "Engine_Condition"]

# Target distribution bar chart
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
df["Engine_Condition"].value_counts().sort_index()\
    .plot(kind="bar", ax=axes[0], color=["steelblue","coral"], edgecolor="black")
axes[0].set_title("Target: Engine_Condition (0=Normal, 1=Maintenance)")

# Feature histograms
df[feats].hist(figsize=(12, 8), bins=30, edgecolor="black", alpha=0.7)
plt.suptitle("Feature distributions")
```

Bivariate - Box Plots & Violin Plots by Class (Cell 23)

```
import seaborn as sns

# Box plots: all features by Engine_Condition
df.boxplot(column=feats, by="Engine_Condition", figsize=(14, 6))
plt.suptitle("Bivariate: Features by Engine Condition")

# Violin plots: 2x3 grid for all 6 features
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
for ax, feat in zip(axes.ravel(), feats):
    sns.violinplot(x="Engine_Condition", y=feat, data=df, ax=ax)
    ax.set_title(feat)
```

Multivariate - Correlation Heatmap & Scatter Plots (Cell 26)

```
# Correlation matrix heatmap
plt.figure(figsize=(9, 7))
sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap="coolwarm",
            center=0, square=True, linewidths=0.5)
plt.title("Multivariate: Correlation matrix")

# Feature-target correlation bar chart
corr_with_target = df.corr()["Engine_Condition"].drop("Engine_Condition").abs()
corr_with_target.sort_values().plot(kind="barh", figsize=(8, 4))
plt.title("Absolute feature-target correlation")
```

A.3 Data Preparation Script - prep.py (Excerpt)

Representative excerpt from prep.py. Full script: see submitted notebook Cell 32.

```
from sklearn.model_selection import train_test_split

COLUMN_RENAME = {
    "Engine rpm": "Engine_RPM",
    "Lub oil pressure": "Lub_Oil_Pressure",
    "Fuel pressure": "Fuel_Pressure",
    "Coolant pressure": "Coolant_Pressure",
    "lub oil temp": "Lub_Oil_Temperature",
    "Coolant temp": "Coolant_Temperature",
    "Engine Condition": "Engine_Condition"
}

df = df.rename(columns=COLUMN_RENAME).drop_duplicates()
df = df.fillna(df.median(numeric_only=True))
df["Engine_Condition"] = df["Engine_Condition"].astype(int)

X = df[FEATURES]
y = df[TARGET]
Xtrain, Xtest, ytrain, ytest = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

A.4 Model Training Code - train.py (Excerpt)

Representative excerpt showing the Gradient Boosting training and MLflow logging block. Full train.py: see submitted notebook Cell 50.

```
import mlflow, mlflow.sklearn, joblib
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import RandomizedSearchCV
from sklearn.metrics import f1_score, accuracy_score, roc_auc_score

gb_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", GradientBoostingClassifier(random_state=42))
])
gb_params = {
    "clf__n_estimators": [50, 100],
    "clf__max_depth": [3, 5],
    "clf__learning_rate": [0.01, 0.1]
}

with mlflow.start_run(run_name="gradient_boosting"):
    gs = RandomizedSearchCV(gb_pipeline, gb_params,
                            n_iter=4, cv=3, scoring="f1", random_state=42)
    gs.fit(Xtrain, ytrain)
    gb_best = gs.best_estimator_
    mlflow.log_params(gs.best_params_)
    y_pred = gb_best.predict(Xtest)
    mlflow.log_metrics({
        "test_accuracy": accuracy_score(ytest, y_pred),
        "test_f1": f1_score(ytest, y_pred, zero_division=0),
        "test_roc_auc": roc_auc_score(ytest, gb_best.predict_proba(Xtest)[:,1])
    })
    mlflow.sklearn.log_model(gb_best, "model_gb")
```

A.5 MLflow Run Comparison Log (Representative Sample)

The following table shows the MLflow run comparison output used for automated model selection. Full run logs are available in the mlruns/ directory of the submitted project.

Table A1: MLflow Run Comparison - Top 3 Runs by Test F1

Run Name	Best Parameters	test_f1	test_accuracy	test_roc_auc
gradient_boosting	n=100, d=3, lr=0.01	**0.7767**	**0.6493**	**0.6899**
adaboost	n=50, lr=0.01	0.7733	-	-
random_forest	n=100, d=10, mss=5	0.7570	0.6593	0.6971

Table A1: MLflow automated run comparison - gradient_boosting selected as best model by test F1

A.6 Full Classification Report - Best Model (Notebook Output)

Exact output from the notebook for the Gradient Boosting best model on the held-out test set:

```
Best model: gradient_boosting (by test F1)

--- Best model performance (test set) ---
              precision    recall  f1-score   support

      Normal       0.66      0.11      0.18      1444
 Maintenance       0.65      0.97      0.78      2463

    accuracy                           0.65      3907
   macro avg       0.65      0.54      0.48      3907
weighted avg       0.65      0.65      0.56      3907

Confusion matrix (test set):
[[  155  1289]
 [   81  2382]]

============================================================
MODEL BUILDING SUMMARY
============================================================
Best model (by test F1): gradient_boosting
Test accuracy:  0.6493
Test F1-score:  0.7767
Test ROC-AUC:   0.6899
------------------------------------------------------------
Best model registration: Saved locally only (set HF_TOKEN to upload)
============================================================
```

A.7 Additional Error Analysis

The 1,289 false positives (Normal engines flagged as Maintenance) represent the most significant operational issue with the current model. Analysis of the confusion matrix reveals a strong bias toward predicting the Maintenance class - a direct consequence of the 63%/37% class imbalance and the use of F1-score as the tuning objective without balancing class weights.

Recommended next steps to address this in subsequent experiments:

11. **Threshold tuning:** Scan classification thresholds from 0.3–0.7 on the validation set and select the value that optimises a cost-weighted metric (e.g., penalise FP and FN asymmetrically based on inspection cost vs. repair cost)
12. **Class weighting:** Re-train with class_weight='balanced' or manually weighted classes to reduce the model's propensity to over-predict Maintenance
13. **SMOTE:** Apply Synthetic Minority Over-sampling Technique to the training set to synthetically balance the Normal class before fitting
14. **Precision-Recall curve:** Plot the full precision-recall curve for each model to inform threshold selection beyond the default 0.5 cutoff

These experiments are planned for the final project report and are expected to materially reduce the false positive rate while maintaining the high Maintenance recall achieved in this iteration.